WOS - Object-Oriented Modeling and UML: summary.

1. OS 01 Object Orientation Good Practices

1. Conceptual Modeling

What it is: Captures ideas and concepts to allow other activities, reasoning and communication about a system.

When to use it: At any time during the software development process, starting from the very beginning.

Tool: Utilizes UML (Unified Modeling Language) as a collection of conceptual modeling notations.

2. Organize Software into Loosely Coupled Layers.

* Design software systems in distinct layers (e.g. User interface, Application logic, Persistent store, Communications, Operating System).
* Each layer should have well-defined responsibilities and minimal direct dependency on lower-layer implementation details to enhance maintainability and modularity.

3. Prefer Aggregation over Inheritance.

* In class-based languages, there are two primary ways to achieve reuse: Inheritance (white-box reuse, fixed at compile time) and Aggregation (black-box reuse, dynamic at runtime).
* Best practice: Prefer aggregation over inheritance because it offers greater flexibility, lower coupling, and allows object behavior and composition to change dynamically at runtime.

4. Program to an Interface or Abstraction.

One way manage complexity and coupling is to depend on interfaces rather than concrete implementations.

5. Use identifiers that improve Readability and Maintainability

Clear descriptive naming conventions for variables, methods, classes and components.

6. Testing with Executable Unit Test Cases.

CamScanner

1.05.07. Introduction to the Unified Modeling Language (UML)

1. Overview of UML.
   * UML is an object-oriented modeling language (or more precisely, a collection of related notations and conceptual modeling standards).
   * It provides a standardized way to visualize, specify, construct, and document the architect of a software-intensive system.

2. Brief Timeline of OO and UML
   * 1960s: Birth of initial Object-Oriented (OO) ideas.
   * 1970s: Maturing of OO concepts and early languages
   * 1980s: Proliferation of diverse OO methods and notations
   * 1990s: The "Method Wars" (Industry fragmentation due to competing methodologies).

3. UML Building Blocks & Modeling Elements.
   * Modeling elements: Building blocks used to construct conceptual descriptions, categorized into shaping, structural, behavioral, organizational and annotational elements.
   * Diagrams: Graphical representations of systems (e.g. use case diagrams, Class diagrams, Object Diagrams, Interaction (Sequence Diagrams).
   * Views: A collection of diagrams that describe one particular aspect of a system or present the system from a specific stakeholder perspective.

4. The three Software-Engineering Modeling Perspective.
   * Analysis Perspective: Focuses on understanding the real-world problem domain and user requirements
   * Specification Perspective: Focused on software interfaces, types, and the public responsibilities or obligations of classes.
   * Implementation Perspective: Focused on actual code realization, algorithms, data structures, and concrete software components

5. Best practices and Guidelines

The primary purpose of diagrams is to effectively communicate, visualize and reason about system, not just to draw pretty pictures.

1.08.08 - Basic Use Cases.

1. Actors.

* Definition: Actors are entities (humans, hardware devices, or other software systems) outside the system being build that need to interact with it.

* Modeling Actors: Actors are identifiers (sets of instances) that play specific roles in relation to the system. Tips for finding actors include asking who or what uses the main functionality.

2. Use cases.

* Definition: Ideas or descriptions that represent what users need to accomplish, independent of specific technical implementations. Each use case captures a user-visible goal.

* Use cases vs System Interactions: Use cases focus on what goals need to be achieved and why, whereas system interactions describe the exact step-by-step dialogue of how users and the system interact.

3 Use case Relationships.

* Extends Dependency (<<extend>>): Defines a use case that represents an optional or specialized variation of another base use case.

* Includes Dependency (<<include>>): Defines how one use case reuses or reuses the behavior and goal defined in another common use case.

* Generalization: Defines a specialized use case as a subset of a more general use case in terms behavior and structure.

4. Use Case Diagrams & Best Practices.

* Components: Consist of actors, use cases, and the relationships connecting them.

* Well-Structured Diagrams: A well-structured diagram focuses on communicating one clear aspect of the system's functionality without becoming cluttered.

* Tips for Modeling: Start with actors first, look at the each actor each needs to perform, establish goals, give each diagram a meaningful name.

1.05.09. Objects, Classes, and Basic Class Diagrams.

1. Introduction to Objects and Classes.
   - Objects & classes: Objects are run time building blocks; classes are blueprints defining their structure and behavior.
   - three perspectives: Handled through Analysis (real-world concepts), Specification (interfaces / obligations), and Implementation (code structures).

2. Syntax and Components.
   - Class syntax: Box with compartments for name (singular noun), Attributes, and Operations.
   - Object syntax: Formatted as instance name: class name.
   - Attributes & Operations: Characteristics and services defined with specific visibility, types, and multiplicity.

3. Relationships in Class Diagrams.

- Generalization: "is-a" inheritance relationship between classes.
- Associations: Structural connections between classes (uni-directional, bi-directional, or N-ary) with navigation and multiplicity constraints (e.g., 1, *, 1..*).
- Aggregation & composition: Specialized "is-part-of" relationships. Aggregation is loose/shared; composition is strict with owned lifecycles.
- Dependencies: Weaker dashed arrows showing that a component "uses" or is affected by another.

1. OOAD - UML Quick Reference Guide.

1. General UML Best Practices for analysis.

Perspective: Maintain an analysis mindset by modeling the real-world problem domain rather than software implementation details.

Iterative discovery: Use diagrams like paragraphs or essays to communicate focused ideas and allow model incompleteness to drive further stakeholder exploration.

2. Use Case Diagrams.

* Use cases: Include actors, goal labels, generalization, extend/sub-goals, and extend vs dependencies.

* Class Diagrams: Cover classes, attributes, associations, multiplicities, aggregation (shared), and composition (owned lifecycle).

3. Activity & Interaction Diagrams.

Activity diagrams: Model process flows using actions/decision/merge, fork/join, object flows, and signals.

Interaction Diagrams: Communication diagrams focus on structural link and messages; sequence diagrams emphasize time-ordered execution and message type.

4. State charts.

States & Transitions: Describe life cycles via states, transitions with guards/trigger/actions, and entry/exit activities.